

Vite & Gourmand

Documentation de gestion de projet

ECF TP Développeur Web et Web Mobile, Studi

Élément	Valeur
Auteur	Arthur Gatoux, FastDev
Outil de gestion de projet	Notion, espace « Vite Gourmand, ECF »
Dépôt public	github.com/arthurgatoux/vite-gourmand
Application déployée	vite-gourmand-ten.vercel.app
Version	1.2, 20 septembre 2026

1. Une conduite de projet structurée

Vite & Gourmand rassemble plusieurs enjeux : catalogue de menus, commande événementielle, gestion de quatre rôles, base relationnelle et base NoSQL, sécurité, accessibilité et déploiement. Il fallait une organisation capable de garder ces sujets liés, sans perdre la vision du besoin initial.

J'ai choisi Notion comme espace de pilotage. Il centralise le Backlog fonctionnel, les choix documentés et les traces de réalisation. GitHub, Supabase, MongoDB Atlas et Vercel complètent ce dispositif. Chaque outil intervient à une étape précise, ce qui évite de disperser les informations.

La règle de travail était simple : une fonctionnalité n'était considérée comme aboutie qu'après développement, contrôle du comportement attendu et vérification de ses éventuels effets sur les rôles, les données ou l'interface. Cette méthode prend un peu plus de temps au départ, mais elle évite les corrections en cascade.

2. Organisation dans Notion

L'espace Notion « Vite Gourmand, ECF » repose principalement sur un Backlog et une Documentation. Le premier sert à organiser le travail fonctionnel. La seconde conserve les éléments nécessaires à la compréhension du projet : décisions techniques, modèle de données, diagrammes, sécurité, déploiement et retours de tests.

Support	Utilité
Backlog	Il rassemble 50 user stories réparties dans 10 épics. Chaque ticket précise son sujet, sa priorité, son statut et, lorsque nécessaire, la branche Git concernée.
Documentation	Elle explicite les choix qui ne sont pas visibles dans l'interface : architecture, données, sécurité, accessibilité, médias et déploiement.
GitHub	Le dépôt public garde la trace du code, des commits, des branches et des Pull Requests.

Cette organisation sépare volontairement le besoin fonctionnel, la décision technique et l'historique du code. Cela rend le projet plus lisible pour une personne qui ne l'a pas suivi au quotidien.

3. Découpage fonctionnel du Backlog

Les 50 user stories sont regroupées en 10 épics. Ce découpage permet d'avancer par ensembles cohérents, sans transformer l'application en un gros bloc difficile à tester. Une branche ou une Pull Request peut alors rester concentrée sur un objectif clair.

Epic	Périmètre fonctionnel
E1	Administration et gestion des comptes employés
E2	Catalogue public, vue des menus et filtres dynamiques
E3	Gestion des menus, plats et horaires
E4	Authentification, profils et contrôle des accès
E5	Espace utilisateur : commande, suivi, modification et avis
E6	Espace employé : traitement des commandes, avis et matériel

Epic	Périmètre fonctionnel
E7	Statistiques MongoDB et tableau de bord administrateur
E8	Qualité, responsive, accessibilité et tests
E9	Sécurité, RLS et règles métier côté serveur
E10	Déploiement, livrables et documentation

4. Déroulement du projet

Le projet a été mené dans un ordre volontairement progressif. La conception et les données sont venues avant l'intégration complète. Ensuite, les parcours publics et utilisateurs ont été construits, puis les espaces métier. Les tests et le déploiement ont permis de confirmer que l'ensemble tenait debout en dehors de l'environnement local.

Étape de travail	Résultat produit
Cadrage	Dépôt GitHub public, analyse des besoins, organisation Notion et README initial.
Conception	Maquettes Figma, charte graphique, modèle relationnel, structure NoSQL, diagrammes et dispositions RGPD.
Installation	Projet Next.js, Supabase, MongoDB Atlas, variables d'environnement et scripts SQL.
Développement	Parcours public, commande, espaces utilisateur, employé et administrateur.
Sécurisation	RLS, rôles, validations serveur, headers HTTP et emails transactionnels.
Qualité	Tests par rôle, corrections UX, responsive et contrôle du build.
Publication	Vercel, variables de production, domaine Resend et recette fonctionnelle en ligne.
Livrables	Manuel, charte graphique, documentation de gestion, documentation technique et copie ECF.

5. Une priorité donnée aux fondations

Les sujets les plus structurants ont été traités avant les améliorations de confort. L'authentification, le modèle de données, le catalogue et la commande étaient prioritaires, car les écrans d'administration ou les statistiques dépendent de ces bases.

Les exigences de sécurité et d'accessibilité ont été intégrées pendant le développement. La Row Level Security, les contrôles de rôle, les validations côté serveur, les contrastes et la navigation clavier ne sont pas des finitions. Ils conditionnent la fiabilité réelle de l'application.

6. Workflow Git et mise en production

Le dépôt public suit le workflow demandé dans le cahier des charges. La branche main représente la version de production. La branche develop sert à intégrer les fonctionnalités avant leur publication. Chaque sujet fonctionnel est développé dans une branche feature dédiée.

Moment	Pratique appliquée
Préparation	Création d'une branche feature liée à un besoin identifié.
Développement	Travail limité à un objectif cohérent, avec migration ou note technique si nécessaire.
Contrôle	Tests fonctionnels, vérification des droits, du rendu mobile et du build.
Intégration	Pull Request vers develop après validation du comportement.
Publication	Fusion de develop vers main, puis déploiement de production sur Vercel.

La fusion de develop vers main a été effectuée le 17 septembre 2026 par la Pull Request n°55.
L'application a ensuite été vérifiée directement sur son URL de production. Le workflow a donc servi de garde fou concret avant mise en ligne.

7. Qualité et correction des anomalies

Une grille de 41 cas de test couvre les parcours visiteur, utilisateur, employé et administrateur, ainsi que plusieurs points transversaux. Les tests n'ont pas servi seulement à confirmer ce qui fonctionnait. Ils ont permis de trouver des erreurs moins visibles, puis de les corriger avant la livraison.

Constat	Correction apportée
Routes publiques bloquées	Le middleware a été ajusté afin que les pages /menus et /contact restent accessibles sans connexion.
Navigation mobile incomplète	Un menu burger a été ajouté sous 640px en réutilisant un composant déjà présent.
Création de commande trop permissive	La policy RLS a été renforcée pour vérifier que utilisateur_id correspond à auth.uid().
Délai de commande absent	La règle a été ajoutée dans une Server Action, côté serveur.
Build Vercel bloquant	Les erreurs TypeScript et la configuration de cache incompatible avec les routes authentifiées ont été corrigées.

Certaines différences mineures existent entre les écrans Figma et l'application intégrée. Elles concernent surtout les tailles intermédiaires, les contenus réellement issus de la base et la navigation clavier. La direction visuelle reste la même. Les adaptations ont été retenues lorsqu'elles amélioraient l'accessibilité, le responsive ou la compréhension des informations dynamiques.

8. Jalons et preuves de réalisation

L'avancement du projet s'appuie sur des résultats observables. Chaque jalon doit laisser une preuve : document, script SQL, branche, Pull Request, test ou version déployée. Cela permet de montrer ce qui a été fait, plutôt que de s'en tenir à une déclaration d'avancement.

Jalon	Preuve disponible
Conception	MCD, diagrammes, charte, maquettes Figma, documentation RGPD et structure MongoDB.
Parcours métier	Espaces visiteur, utilisateur, employé et administrateur, commande et dashboard NoSQL.
Sécurité	Audit RLS, Server Actions, headers HTTP et contrôles de rôle.
Recette	41 cas de test, anomalies tracées et correctifs vérifiés.
Publication	Déploiement Vercel, domaine Resend vérifié et recette sur l'URL publique.
Livrables	README, manuel utilisateur PDF, charte graphique PDF et documentation de gestion.

9. Bilan méthodologique

Cette méthode relie le besoin client, le travail de développement et les éléments de preuve. Notion facilite la lecture fonctionnelle, GitHub conserve l'historique du code, Supabase et MongoDB Atlas matérialisent les décisions liées aux données, tandis que Vercel valide la capacité à livrer une application disponible.

Le choix le plus important reste la protection des règles métier hors du seul navigateur. Les rôles, les droits d'accès, les prix de commande, les transitions de statut, l'historique et les statistiques sont contrôlés côté serveur ou en base. Le projet devient alors plus robuste, et surtout plus simple à défendre lorsqu'on regarde ce qui se passe derrière les écrans.